TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 02

ỨNG DỤNG CỦA MẠCH SỐ

Họ và tên	Võ Đông Đô	**MSSV**	25520311
Lớp	IT012.R11.1	**Ngày thực hành**	5/10/2026
Tự chấm	10/10	**Link git**	https://github.com/Ryan-0734/IT012_Lab01_25520311.git

1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 4 (mạch số, mạch tổ hợp, mạch tuần tự, latch, flip-flop, thanh ghi) và Chương 5 (bộ cộng, ALU, bộ chọn, bộ so sánh, bộ giải mã, tập thanh ghi) thành một bài toán xuyên suốt: **tự xây một datapath 8 bit của CPU thu nhỏ**, từng khối một, theo chu trình **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Thiết kế và dựng bộ cộng/trừ từ cổng luận lý theo quy trình 4 bước (bảng chân trị → biểu thức → tối ưu → sơ đồ), đọc được các **cờ** Z, N, C, V và phát hiện **tràn số**.
- Dựng **ALU 8 bit** từ bộ chọn (MUX), bộ cộng/trừ và cổng luận lý; dùng ALU để so sánh hai số và chỉ ra **giới hạn** của quy tắc so sánh bằng phép trừ.
- Phân biệt **latch** (tích cực theo mức) với **flip-flop** (tích cực theo cạnh) bằng dạng sóng; tự dựng D latch và ghép hai latch thành D flip-flop.
- Giải thích vì sao thanh ghi dùng flip-flop và vì sao vòng hồi tiếp "thanh ghi → ALU → thanh ghi" vẫn hoạt động đúng.
- Dựng **tập thanh ghi 4 × 8 bit**, ghép với ALU thành đường dữ liệu và điều khiển bằng tay để chạy một chương trình ngắn (C = A + B, hoán đổi hai thanh ghi).

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen bus nhiều bit, mạch con, clock; kiểm tra môi trường	15 phút	
TN 1	Bộ cộng/trừ 4 bit	45 phút	2,0
TN 2	ALU 8 bit và bộ so sánh	45 phút	1,5
TN 3	Mạch tuần tự: D latch, D flip-flop	45 phút	3,0
TN 4	Tập thanh ghi và đường dữ liệu 8 bit	60 phút	3,0
Kết	Tổng kết, phản tư, nộp bài	15 phút	0,5
Tổng		**225 phút**	**10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕ B. Số thập lục phân viết 0x3C. Với bus nhiều bit, A[7] là bit trọng số cao nhất (MSB). Số có dấu dùng biểu diễn **bù 2** (Chương 2).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.
- Các mạch con (subcircuit) tạo ở thí nghiệm trước được **dùng lại** ở thí nghiệm sau. Đặt tên đúng quy ước và không xoá.

PHẦN 0 — Làm quen bus, mạch con và clock (15 phút)

Sinh viên đã dùng Logisim ở Lab 01; phần này chỉ bổ sung các kỹ năng mới cho mạch nhiều bit và mạch tuần tự. Giảng viên hướng dẫn nhanh theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab02_<MSSV>.circ**	Mỗi thí nghiệm dùng các circuit riêng trong **cùng một file** (Project ▸ Add Circuit); tên circuit theo quy ước ở đầu mỗi thí nghiệm.
☑	Đặt Input Pin / Output Pin **nhiều bit** (thuộc tính Data Bits); đổi từng bit bằng Poke tool	Pin 8 bit hiển thị 8 ô bit. Dùng **Probe** (thư viện Wiring) và đổi thuộc tính **Radix** để đọc ở dạng nhị phân, thập lục phân, thập phân có dấu.
☑	Dùng **Splitter** để tách bus thành từng dây và ghép các dây thành bus	Fan Out = số dây, Bit Width In = độ rộng bus. Kiểm tra thứ tự bit (bit 0 là LSB).
☑	Tạo **mạch con**: dựng một circuit, rồi kéo circuit đó từ ngăn Explorer vào circuit khác như một linh kiện	Vị trí các pin trong circuit quyết định vị trí các chân của linh kiện; đặt nhãn pin rõ ràng.
☑	Dùng linh kiện có sẵn: **Plexers** (Multiplexer, Decoder), **Arithmetic** (Adder, Subtractor), **Memory** (D Flip-Flop, Register)	Chọn linh kiện rồi chỉnh Data Bits, Select Bits ở ngăn thuộc tính (góc dưới bên trái).
☑	Điều khiển mạch tuần tự: đổi tay một Input Pin **CLK**; dùng **Simulate ▸ Reset Simulation** để đưa mạch về trạng thái ban đầu	Cạnh lên xảy ra khi Poke CLK từ 0 lên 1. Với linh kiện Clock (Wiring), mỗi lần Simulate ▸ Tick Once đổi Clock một nửa chu kỳ.
☑	(Tuỳ chọn) Xem dạng sóng: **Chronogram** (Logisim-evolution) hoặc cửa sổ **Simulate ▸ Logging** (Logisim 2.7.x)	Tuỳ phiên bản; không bắt buộc, dùng để kiểm tra lại Bảng 10 ở TN 3.
☑	Chụp màn hình mạch và dán vào khung ảnh của file này	Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra và giá trị đang hiển thị.

Phiên bản Logisim đang dùng	logisim-win-2.7.1

Đọc màu dây và giá trị trong Logisim

Xanh lá sáng = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = chưa xác định / chưa được điều khiển · **Đỏ** = lỗi/xung đột · **Cam** = hai đầu dây lệch số bit.

Ở mạch **tuần tự**, ngay sau khi mở file hoặc Reset Simulation, giá trị đang lưu có thể **chưa xác định** và hiển thị là **x** (hoặc dây xanh dương) cho đến khi mạch được ghi lần đầu. Đó là hành vi bình thường, không phải lỗi; nếu phiên bản Logisim của Sinh viên hiển thị khác, hãy ghi lại điều quan sát được.

Bước 0 – Kiểm tra nhanh. Trong circuit **Warmup**, đặt hai Input Pin 8 bit **A**, **B**, một **Adder** (Arithmetic, Data Bits = 8), một Output Pin 8 bit **S** và một Probe. **Dự đoán** S (viết hex) bằng tay trước khi Poke, rồi đọc S ở dạng thập phân **không dấu** và **có dấu** (bù 2).

A	B	S = A + B (hex)	S không dấu	S có dấu (bù 2)
0x20	0x0F	0x2F	47	47
0x7F	0x01	0x80	128	-128
0xFF	0x01	0x00	0	0
0x64	0x64	0xC8	200	-56

Circuit Warmup chỉ để làm quen, không nộp. Hãy để ý: cùng một dãy bit, hai cách đọc cho hai giá trị khác nhau; Thí nghiệm 1 sẽ trả lời "phần cứng báo cho phần mềm biết kết quả có đáng tin hay không" bằng cách nào.

THÍ NGHIỆM 1 — Bộ cộng/trừ 4 bit và các cờ trạng thái

Thời lượng gợi ý: 45 phút · Circuit: **FA, ADDSUB4** · Điểm: 2,0

> **Tình huống**
> Phép cộng là phép toán nền tảng của ALU: phép trừ, tăng/giảm và so sánh đều quy về bộ cộng. Sinh viên sẽ tự xây bộ cộng 4 bit từ cổng luận lý, biến nó thành bộ cộng/trừ, rồi tự thiết kế mạch **phát hiện tràn số** để biết khi nào kết quả 4 bit không còn đúng, thông tin mà CPU thật luôn cung cấp dưới dạng các cờ.

A. Bộ cộng toàn phần (FA) từ cổng luận lý

Bước 1.1 – Dự đoán. Cộng ba bit A + B + Cin cho kết quả từ 0 đến 3, viết được bằng hai bit (Cout S). Điền hai cột "dự đoán" của Bảng 1 bằng phép cộng số học, **chưa** dựng mạch.

DỰNG MẠCH **Bước 1.2.** Trong circuit **FA** (circuit mới), đặt 3 Input Pin **A, B, Cin** và 2 Output Pin **S, Cout**. Dựng theo slide: S = A ⊕ B ⊕ Cin; Cout = A·B + Cin·(A ⊕ B), dùng chung cổng A ⊕ B cho cả hai ngõ ra.

QUAN SÁT Dùng Poke tool (hoặc Analyze Circuit ▸ Table) để điền hai cột "đo".

Bảng 1. Bộ cộng toàn phần FA

A	B	Cin	S (dự đoán)	Cout (dự đoán)	S (đo)	Cout (đo)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	1

Câu 1.1 (Phân tích). Cột Cout chính là hàm "có từ 2 ngõ vào trở lên bằng 1" mà slide Chương 4 dùng làm ví dụ thiết kế. Viết SoP tối giản của Cout (gom bìa Karnaugh 3 biến nếu cần) và tính C(B). So sánh với dạng Cout = A·B + Cin·(A ⊕ B) khi cổng A ⊕ B đã có sẵn để tạo S: tổng số cổng của cả mạch FA ở mỗi cách là bao nhiêu?

Dạng SoP của Cout: Cout = A.B + A.Cin + B.Cin
C(B) = 3 + 2 + 2 + 2 = 9
dạng Cout = A·B + Cin·(A ⊕ B) khi cổng A ⊕ B đã có sẵn để tạo S tổng có 2 (XOR) + 2 (AND) + 1 (OR) = 5 (cổng). Còn Tổng số cổng của cả mạch FA ở dạng SoP tối giản cần 6 cổng : 2 cổng XOR, 3 cổng AND, 1 cổng OR.

Câu 1.2 (Phân tích). Đếm **số tầng cổng** (độ sâu) của mạch FA ở bước 1.2 trên hai đường đi: (a) từ **Cin** đến **Cout**; (b) từ **A hoặc B** đến **Cout**. Ghi cách đếm.

a) Độ sâu 2 tầng.
Cách đếm: Đi từ ngõ Cin phải đi qua cổng AND để tạo thành tầng 1. Sau đó, kết quả từ cổng AND này đi tiếp qua cổng OR (để cộng với nhánh $A \cdot B$) tạo thành tầng thứ hai để ra đến Cout.

b) Độ sâu 3 tầng:
Cách đếm: Đi từ ngõ A hoặc B phải đi qua cổng XOR trước là tầng thứ nhất A $\oplus$ B, kế tiếp ta thấy kết quả từ ngõ ra cổng XOR đi đến cổng AND để cùng với Cin tạo thành tầng 2. Sau đó, kết quả từ AND này sẽ đi tiếp qua cổng OR (để cộng với nhánh A.B) tạo thành tầng 3. Lí do chọn đường dài nhất vì độ sâu, ta luôn chọn đường dài nhất dù A và B đi đến qua nhánh A.B chỉ mất 2 tầng.

B. Ghép thành bộ cộng 4 bit nối tiếp

DỰNG MẠCH **Bước 1.3.** Trong circuit **ADDSUB4**: Input Pin **A** (4 bit), **B** (4 bit), **SUB** (1 bit); Output Pin **S** (4 bit), **C** (1 bit). **Lần này chưa dùng SUB:** nối 4 mạch con FA liên tiếp (Cout của tầng i vào Cin của tầng i + 1; Cin của tầng 0 nối hằng 0). Dùng Splitter để tách A, B và gộp S. Đặt Probe (hoặc Output Pin tạm) cho ba dây nhớ **C1, C2, C3** giữa các tầng.

DỰ ĐOÁN Với ba phép cộng ở Bảng 2, dự đoán S, các bit nhớ và **số tầng mà bit nhớ đã lan qua** (bắt đầu đếm từ khi C1 = 1).

Bảng 2. Bit nhớ lan truyền trong bộ cộng 4 bit (Cin = 0)

A	B	S (nhị phân)	C1	C2	C3	Cout	Số tầng bit nhớ lan qua
0001	0001	0010	1	0	0	0	1
0111	0001	1000	1	1	1	0	3
1111	0001	0000	1	1	1	1	4

Câu 1.3 (Đánh giá). Với bộ cộng nối tiếp n bit, bit nhớ có thể phải đi qua tất cả các tầng. Dựa vào Câu 1.2, điền số tầng cổng của đường đi dài nhất (từ A[0]/B[0] đến Cout) ở Bảng 3 và rút ra công thức theo n. Nhận xét: vì sao CPU 32/64 bit thực tế không dùng thuần bộ cộng nối tiếp?

Bảng 3. Độ sâu của bộ cộng nối tiếp

n (số bit)	Số tầng FA mà bit nhớ đi qua	Số tầng cổng đến Cout
4	4	9
8	8	17
32	32	65
n	n	2n + 1

Số tầng FA mà n bit nhớ đi qua: n
Tổng số tầng cổng cho mạch n bit là: 2n + 1
Vì Công thức độ sâu (số tầng cổng): Đường đi dài nhất của tín hiệu bắt đầu từ đầu vào A_0, B_0 của tầng đầu tiên đi đến ngõ ra Cout cuối cùng. Tại tầng FA đầu tiên (từ A_0/B_0 đến $Cout_0$), tín hiệu đi qua 3 tầng cổng. Tại các tầng FA tiếp theo, tín hiệu truyền từ Cin đến Cout đi qua 2 tầng cổng. (tổng số tầng cổng cho mạch n bit là: 3 + 2.(n-1) = 2n + 1)
Nhận xét : Đối với bộ cộng nối tiếp thuần túy, độ trễ thời gian (thể hiện qua số tầng cổng) tỉ lệ thuận bậc nhất ($O(n)$) với số lượng bit n. Nếu áp dụng trực tiếp cho CPU 32-bit hay 64-bit thực tế, tín hiệu sẽ phải chờ đi qua quá nhiều tầng cổng logic (lên tới 65 tầng cho 32-bit và 129 tầng cho 64-bit). Thời gian trễ này quá lớn, không thể đáp ứng được các mức xung nhịp cao.

C. Biến thành bộ cộng/trừ

Câu 1.4 (Hiểu). A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2). Từ đó giải thích vì sao chỉ cần thêm 4 cổng XOR (mỗi cổng nhận B[i] và SUB) và nối SUB vào Cin của tầng 0 là bộ cộng thành bộ cộng/trừ. Điều gì xảy ra ở hai trường hợp SUB = 0 và SUB = 1?

Vì mạch đang xử lí có 4 bit, mà ta thực hiện phép tính A – B nên chọn B để phân tích, do B có 4 đường tín hiệu rời rạc (B0 – B3) nên cần đúng 4 cổng XOR mắc vào 4 đường dây của B.
Do A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2).
SUB = 0
Tín hiệu B[i] đi qua cổng XOR sẽ giao với 0. Theo logic, $B[i] \oplus 0 = B[i]$, do đó dữ liệu đầu vào B được giữ nguyên.Cin của bộ cộng tầng 0 được nối trực tiếp với SUB nên $Cin = 0$. Mạch thực hiện phép tính tổng quát là $A + B + 0 = A + B$, đóng vai trò là một bộ cộng bình thường.
SUB = 1
Tín hiệu B[i] đi qua cổng XOR sẽ giao với 1. Theo logic, $B[i] \oplus 1 = B'[i]$, do đó toàn bộ các bit của B bị đảo ngược (tạo thành bù 1 của B). Cin của tầng 0 nối với SUB nên $Cin = 1$. Mạch thực hiện phép tính $A + B' + 1$. Vì $B' + 1$ chính là cách biểu diễn số bù 2 của $-B$, biểu thức tổng quát trở thành $A + (-B) = A - B$. Lúc này, mạch hoạt động như một bộ trừ.

DỰNG MẠCH **Bước 1.4.** Thêm vào ADDSUB4: 4 cổng XOR cho B'[i] = B[i] ⊕ SUB; nối SUB vào Cin của tầng 0. Kiểm tra nhanh: SUB = 0, A = 0011, B = 0100 → S = 0111; SUB = 1, A = 0111, B = 0011 → S = 0100.

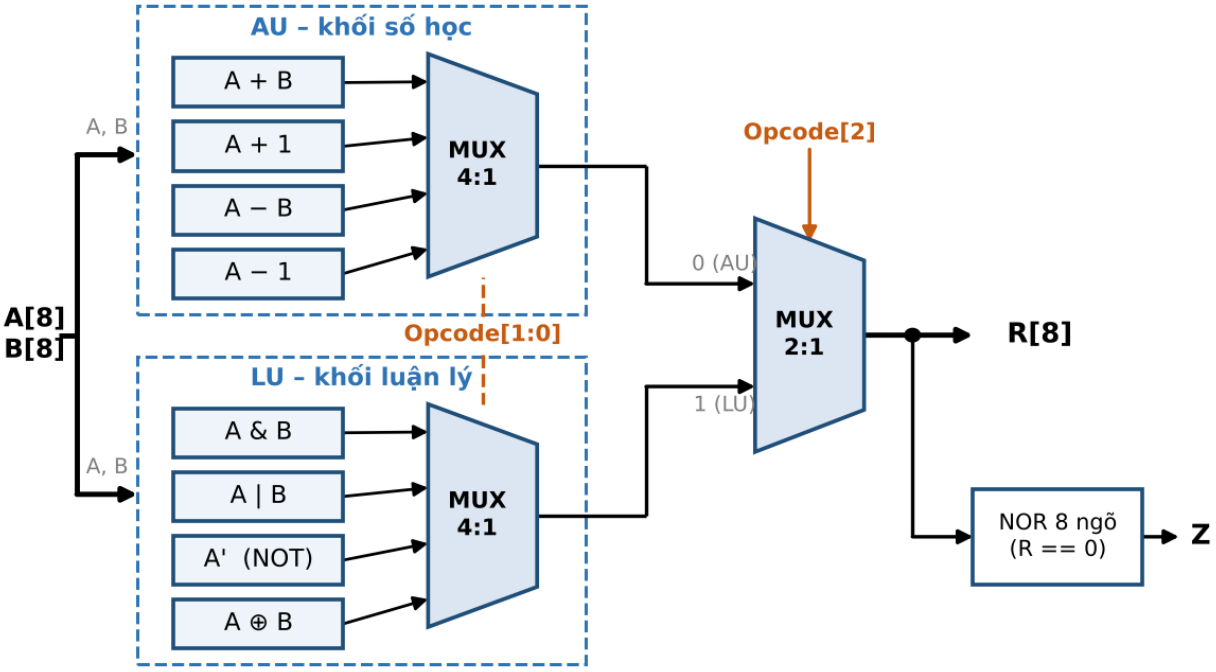
THÍ NGHIỆM 2 — ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?

Thời lượng gợi ý: 45 phút · Circuit: **ALU8, LT** · Điểm: 1,5

Tình huống

Một CPU chỉ cần **một** khối tính toán dùng chung cho nhiều phép toán: ALU. Sinh viên dựng ALU 8 bit theo bảng Opcode ở slide Chương 5 (Hình 1), rồi dùng chính ALU để **so sánh hai số** thay vì thiết kế bộ so sánh riêng, vì bảng chân trị của bộ so sánh hai số 8 bit có 2^16 hàng. Slide nêu quy tắc: "Y = A − B; Y[MSB] = 1 ⇒ A < B; Y[MSB] = 0 ⇒ A > B". Sinh viên sẽ **kiểm tra** quy tắc này có luôn đúng không.

A. Dựng ALU8 theo cấu trúc AU – LU – MUX



Hình 1. *Cấu trúc ALU 8 bit theo slide Chương 5. Opcode[1:0] chọn phép toán trong từng khối; Opcode[2] chọn khối AU hay LU.*

DỰNG MẠCH **Bước 2.1.** Trong circuit **ALU8**: Input Pin **A, B** (8 bit), **OP** (3 bit); Output Pin **R** (8 bit), **Z**. (a) **AU:** hai Adder (A + B, A + 1) và hai Subtractor (A − B, A − 1); hằng 1 lấy từ Constant 8 bit; chọn kết quả bằng Multiplexer 4:1 (8 bit) với Select = OP[1:0] (dùng Splitter tách OP). (b) **LU:** các cổng AND, OR, NOT, XOR có Data Bits = 8; chọn bằng một Multiplexer 4:1 khác với Select = OP[1:0]. (c) Multiplexer 2:1 (8 bit) chọn giữa AU và LU theo OP[2]. (d) **Z** = NOR của 8 bit R (Splitter + cổng NOR 8 ngõ vào).

Gợi ý: ngõ vào số k của Multiplexer được chọn khi Select = k. Hãy đối chiếu thứ tự nối với bảng Opcode ở slide (000: A + B, 001: A + 1, 010: A − B, 011: A − 1, 100: A & B, 101: A | B, 110: A', 111: A ⊕ B).

DỰ ĐOÁN Điền cột "dự đoán" của Bảng 6 bằng tay (hex, 8 bit) **trước khi** Poke.

QUAN SÁT **Ca 1:** A = 0x3C, B = 0x0F. **Ca 2:** A = 0x00, B = 0x01. Điền cột "đo" và cờ Z của Ca 2.

Bảng 6. Kiểm thử ALU8 với 8 Opcode

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
000	A + B	0x4B	0x4B	0x01	0x01	0

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
001	A + 1	0x3D	0x3D	0x01	0x01	0
010	A − B	0x2D	0x2D	0xFF	0xFF	0
011	A − 1	0x3B	0x3B	0xFF	0xFF	0
100	A & B	0x0C	0x0C	0x00	0x00	1
101	A | B	0x3F	0x3F	0x01	0x01	0
110	A'	0xC3	0xC3	0xFF	0xFF	0
111	A ⊕ B	0x33	0x33	0x01	0x01	0

Câu 2.1 (Phân tích). Ở Ca 2, phép nào cho R = 0xFF? Giá trị đó là 255 hay −1? ALU có "biết" điều đó không? Z = 1 ở phép nào, và Z = 1 có đồng nghĩa với "hai toán hạng bằng nhau" không?

Ở Ca 2, các phép cho R = 0xFF là A − B, A − 1 và A'. Giá trị 0xFF có thể được hiểu là 255 nếu xét số không dấu, hoặc −1 nếu xét số có dấu theo biểu diễn bù 2. ALU chỉ xử lý các bit nên không tự biết giá trị đó là 255 hay −1.
Z = 1 ở phép A & B, vì kết quả R = 0x00. Z = 1 chỉ cho biết kết quả phép toán bằng 0, không có nghĩa là hai toán hạng bằng nhau.

Câu 2.2 (Áp dụng). ALU này không có phép "chép" (MOV: R = A). Đề xuất **hai** cách dùng đúng ALU này để có R = A (ghi Opcode và giá trị cần đặt cho B). Cách nào dùng được khi ta **chỉ chọn được thanh ghi** cho A và B, không đặt được hằng số? (Sẽ dùng ở Thí nghiệm 4.)

Có thể dùng hai cách để có R = A:
Cách 1: Chọn Opcode 101 (A | B) và đặt B = 0x00. Khi đó A | 0 = A.
Cách 2: Chọn Opcode 111 (A ⊕ B) và đặt B = 0x00. Khi đó A ⊕ 0 = A.
Nếu chỉ chọn được thanh ghi cho A và B, không đặt được hằng số, thì cả hai cách đều dùng được nếu có một thanh ghi đang chứa giá trị 0x00 để chọn làm B.

B. So sánh hai số bằng ALU

DỰ ĐOÁN Với 6 cặp (A, B) ở Bảng 7, tính R = A − B trên **8 bit**, rồi điền cột "Quy tắc slide kết luận" (dùng <, =, >). Điền hai cột "Thực tế" theo hai cách hiểu: A, B là số **có dấu** (bù 2) và là số **không dấu**.

QUAN SÁT Đặt Opcode = 010 trên ALU8, ghi R, Z, R[7] của từng cặp. Cột **LT đo** để trống, sẽ điền sau Bước 2.3.

Bảng 7. So sánh bằng phép trừ

A	B	R = A − B (đo)	Z	R[7]	Quy tắc slide kết luận	Thực tế (có dấu)	Thực tế (không dấu)	LT đo
0x05	0x07	0xFE	0	1	<	<	<	1
0x09	0x09	0x00	1	0	=	=	=	0
0x64	0x9C	0xC8	0	1	<	>	<	0
0x9C	0x64	0x38	0	0	>	<	>	1
0xC8	0x0A	0xBE	0	1	<	<	>	1
0x0A	0xC8	0x42	0	0	>	>	<	0

Câu 2.3 (Phân tích). Tìm các dòng mà quy tắc slide cho kết luận **sai**. Với mỗi dòng sai, nêu nguyên nhân (gợi ý: phép trừ 8 bit có bị tràn không? A và B đang được hiểu là số có dấu hay không dấu?). Quy tắc ở slide chỉ đáng tin trong điều kiện nào?

Quy tắc slide cho kết luận sai ở 4 dòng: (0x64, 0x9C), (0x9C, 0x64), (0xC8, 0x0A), (0x0A, 0xC8).
Với hai dòng (0x64, 0x9C) và (0x9C, 0x64), nếu xét số có dấu thì phép trừ bị tràn số, làm cho bit R[7] không phản ánh đúng dấu của kết quả thực. Vì vậy không thể chỉ dựa vào R[7] để so sánh số có dấu.
Với hai dòng (0xC8, 0x0A) và (0x0A, 0xC8), khi xét số không dấu, R[7] không phải là bit xác định quan hệ < hay > vì phép trừ có thể xảy ra hiện tượng mượn.
Do đó, quy tắc R[7] = 1 ⇒ A < B, R[7] = 0 ⇒ A > B chỉ đáng tin khi A, B là số có dấu và phép trừ không bị overflow. Khi có overflow hoặc khi xét số không dấu, cần dùng thêm cờ phù hợp để kết luận chính xác.

THIẾT KẾ **Bước 2.2 – Sửa quy tắc cho số có dấu.** Thiết kế mạch **LT** (A < B, số có dấu) chỉ dựa vào ba bit A[7], B[7], R[7] (R = A − B). Gợi ý: nếu A và B cùng dấu thì phép trừ không bao giờ tràn nên LT = R[7]; nếu khác dấu thì LT phụ thuộc dấu của A. Điền Bảng 8, bìa Karnaugh, rồi viết biểu thức tối giản.

Bảng 8. Bảng chân trị của LT(A7, B7, R7)

A7	B7	R7	LT (A < B có dấu)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1

A7	B7	R7	LT (A < B có dấu)
1	0	1	1
1	1	0	0
1	1	1	1

A7 \ B7 R7	00	01	11	10
0	0	1	0	0
1	1	1	1	0

LT tối giản = …	LT tối giản = B7'·R7 + A7·B7' + A7·R7
Số cổng cần dùng (kể cả cổng NOT)	5

Câu 2.4 (Phân tích). Biểu thức LT quen thuộc không? (Gợi ý: so sánh với hàm đa số ở slide Chương 4 và với Cout của FA ở Thí nghiệm 1, trong đó một ngõ vào được lấy bù.) Giải thích bằng lời vì sao có sự giống nhau đó (nhớ lại A − B = A + B' + 1).

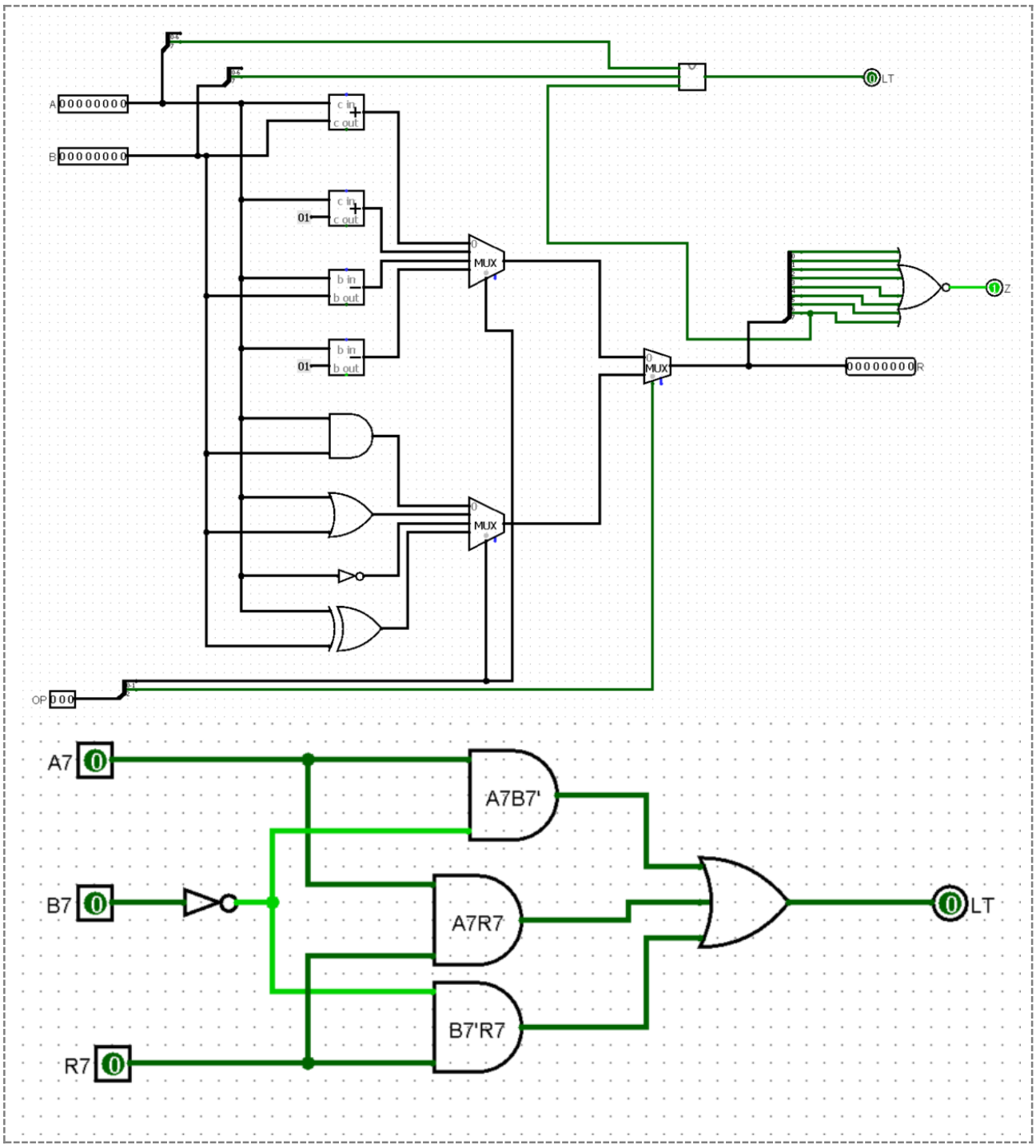
Biểu thức: B7'.R7 + A7.B7' + A7.R7 chính là hàm đa số của 3 biến, có cấu trúc tương tự như ngõ ra truyền nhớ Cout của bộ cộng toàn phần (FA) ở Thí nghiệm 1 nhưng thay ngõ vào B bằng B'. Giải thích: Phép trừ trong ALU thực chất là phép cộng bù hai (A - B = A + B' + 1). Do đó, các biến tham gia quyết định dấu và quan hệ lớn/nhỏ gồm dấu của A (A7), dấu đã đảo của B (B7'), và dấu của kết quả phép trừ (R7). Điều kiện để xác định A < B (có dấu) dựa trên sự áp đảo giá trị giữa ba tín hiệu này, dẫn đến hàm logic có dạng hàm đa số y hệt như mạch tạo nhớ Cout.

DỰNG MẠCH **Bước 2.3.** Dựng mạch con **LT** (ngõ vào A7, B7, R7; ngõ ra LT), nối vào ALU8 (A[7], B[7], R[7]) và điền cột **LT đo** của Bảng 7. LT có sửa được các dòng sai ở Câu 2.3 không?

Câu 2.5 (Đánh giá – mở rộng). Với số **không dấu**, A < B khi nào? (Gợi ý: ý nghĩa của cờ C khi trừ ở Câu 1.6b.) Kiểm tra điều kiện của Sinh viên trên hai dòng 0xC8 và 0x0A của Bảng 7 bằng

cách tính A − B trên 9 bit. Sau đó, so sánh chi phí: bộ so sánh trực tiếp hai số 8 bit cần bảng chân trị bao nhiêu hàng, so với “ALU + LT”? Khi nào vẫn nên dùng bộ so sánh riêng?

Mạch con LT đã khắc phục triệt để hiện tượng tràn số khi so sánh số có dấu, giúp các trường hợp bị kết luận sai trên slide (ví dụ như dòng 3 và dòng 4) nay hiển thị chính xác theo đúng thực tế có dấu.
Với số không dấu: A < B khi và chỉ khi phép trừ A - B sinh ra cờ mượn (borrow/carry out khi trừ).
Kiểm tra trên hai dòng 0xC8 và 0x0A (tính trên 9 bit):
A = 0xC8 = 0 1100 1000_2 (200 hệ 10)
B = 0x0A = 0 0000 1010_2 (10 hệ 10)
Phép trừ 9-bit: 200 - 10 = 190 > 0 (không có mượn âm ra ngoài), do đó A >= B (phù hợp với thực tế không dấu).
So sánh chi phí phần cứng:
Bộ so sánh trực tiếp hai số 8-bit cần bảng chân trị với 2^{16} = 65536 hàng.
Tổ hợp "ALU + LT" tận dụng khối ALU có sẵn và chỉ bổ sung thêm một mạch logic rất nhỏ cho LT, tiết kiệm đáng kể tài nguyên phần cứng.
Khi nào nên dùng bộ so sánh riêng: Khi hệ thống chỉ thuần túy cần chức năng so sánh độ lớn mà không cần các phép toán số học/logic khác của ALU, hoặc khi cần tối ưu hóa tốc độ cao (độ trễ truyền thấp) cho riêng khối so sánh.

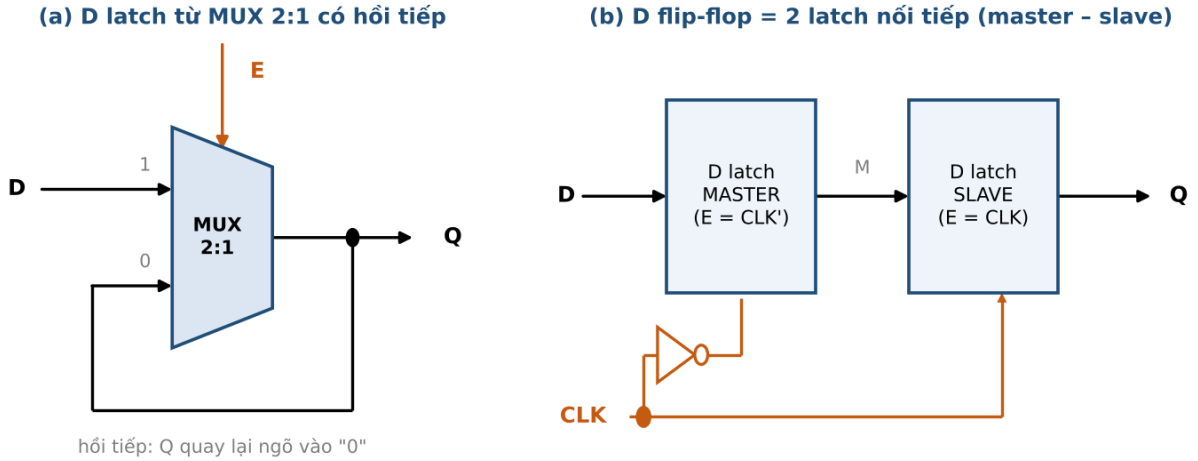

A
00000000
B
00000000
c in
c out
01
b in
b out
MUX
LT
Z
00000000
R
OP
000
A7
B7
R7
A7B7'
A7R7
B7'R7
LT

THÍ NGHIỆM 3 — Mạch tuần tự: D latch và D flip-flop

Thời lượng gợi ý: 45 phút · Circuit: **TN3 (mạch con: DLATCH, DFF_MS)** · Điểm: 3,0

Tình huống

ALU ở Thí nghiệm 2 là mạch tổ hợp: ngõ ra đổi ngay khi ngõ vào đổi và **không nhớ** gì. Muốn CPU giữ lại kết quả để dùng cho lệnh sau, cần **thiết bị lưu trữ** (slide Chương 4). Sinh viên sẽ tự dựng D latch từ bảng trạng thái ở slide, so sánh nó với D flip-flop có sẵn, rồi ghép hai latch thành một flip-flop và tìm hiểu vì sao thanh ghi phải dùng flip-flop.



Hình 2. *(a) D latch dựng từ MUX 2:1 có hồi tiếp; (b) D flip-flop ghép từ hai latch (master – slave).*

A. D latch dựng từ bảng trạng thái

Slide Chương 4 mô tả D latch bằng bảng trạng thái kế tiếp: **Q+ = Q khi E = 0** (giữ), **Q+ = D khi E = 1** (nhận). Đó chính là một **MUX 2:1** mà ngõ vào "giữ" được nối với **ngõ ra Q của chính nó** (Hình 2a).

DỰNG MẠCH **Bước 3.1.** Trong circuit **DLATCH**: Input Pin **D, E**; Output Pin **Q**; một Multiplexer 1 bit (Select = E, ngõ vào 1 = D, ngõ vào 0 = **Q nối ngược về**).

DỰ ĐOÁN Chọn **Simulate ▸ Reset Simulation**, rồi thực hiện lần lượt các thao tác ở Bảng 9 (đổi E và D bằng Poke). Điền Q dự đoán **trước**; nếu Q chưa xác định thì ghi **x**.

QUAN SÁT Sau đó điền cột "Q đo".

Bảng 9. Chuỗi thao tác trên DLATCH

Bước	E	D	Q dự đoán	Q đo
1	0	0	x	x
2	0	1	x	x
3	1	1	1	1
4	0	1	1	1
5	0	0	1	1
6	1	0	0	0
7	1	1	1	1
8	1	0	0	0
9	0	0	0	0

Bước	E	D	Q dự đoán	Q đo
10	0	1	0	0

Câu 3.1 (Hiểu). Vì sao ở bước 1–2 Q chưa xác định dù đã có D? Điều đó nói gì về việc phải **khởi tạo** (reset) thiết bị lưu trữ trong mạch thật?

Ở bước 1 và 2, tín hiệu cho phép E = 0 nên Mux chọn ngõ vào số 0 (nhận tín hiệu vòng ngược từ ngõ ra Q). Do mạch mới khởi động và chưa được ghi giá trị hợp lệ trước đó, đường hồi tiếp không mang mức logic cố định, nên ngõ ra Q ở trạng thái chưa xác định x mặc dù ngõ vào D có thay đổi.
Điều này cho thấy các phần tử nhớ tuần tự không tự động mang giá trị 0 hay 1 khi mới cấp nguồn. Trong mạch thực tế, việc thiết kế tín hiệu Reset là bắt buộc để đưa thiết bị lưu trữ về trạng thái xác định ngay khi khởi động, tránh lỗi hệ thống.

Câu 3.2 (Phân tích). Slide Chương 4 định nghĩa mạch tổ hợp là mạch **không có hồi tiếp**; DLATCH có đường hồi tiếp từ Q về MUX. Khi E = 0, mô tả đường đi của tín hiệu để giải thích vì sao hồi tiếp làm mạch "nhớ". Khi E = 1 thì hồi tiếp còn tác dụng không?

Khi E=0, nên MUX chọn ngõ Q hồi tiếp, nên Q được đưa lại chính ngõ vào và giữ nguyên dù D thay đổi.
Khi E=1, nên MUX chọn D. Hồi tiếp không quyết định giá trị nữa, latch trong suốt và Q theo D.

B. Latch và flip-flop: cùng ngõ vào, khác kết quả

DỰNG MẠCH **Bước 3.2.** Trong circuit **TN3**: Input Pin **D** và **CLK** (đổi bằng Poke, **không** dùng Clock tự chạy); một DLATCH với E = CLK (ngõ ra **Q1**); một **D Flip-Flop** (Memory, kích cạnh lên – Rising Edge) với ngõ vào D, CLK (ngõ ra **Q2**).

DỰ ĐOÁN Bảng 10 cho CLK và D theo từng **khe thời gian** (mỗi cột một khe; tại ranh giới giữa hai khe chỉ đổi **một** tín hiệu: CLK hoặc D, không bao giờ cả hai). Giả sử ban đầu Q1 = Q2 = 0. Điền hàng dự đoán Q1, Q2. Đây là bài "hoàn thành dạng sóng" ở slide Chương 4.

QUAN SÁT Đưa mạch về Q1 = Q2 = 0 (Reset Simulation; D = 0; CLK 0 → 1 → 0). Sau đó Poke từng khe theo bảng và ghi Q1, Q2 đo được. Hàng Q3 để trống đến Bước 3.3.

Bảng 10. Dạng sóng theo khe thời gian

Khe	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CLK	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
D	0	1	1	0	0	1	1	1	1	0	0	1	1	0	0	0
Q1 dự đoán	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q1 đo	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q2 dự đoán	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q2 đo	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q3 đo (DFF_MS)																

Câu 3.3 (Phân tích). Liệt kê các khe mà Q1 ≠ Q2. Với từng khe, giải thích bằng định nghĩa "tích cực theo **mức**" (latch) và "tích cực theo **cạnh**" (flip-flop) ở slide. Trong khoảng CLK = 1, D đổi một lần và Q1 đổi theo, còn Q2 thì không: vì sao?

Các khe thời gian mà Q1 khác Q2 gồm các khe 3, 4, 5 và 11, 12, 13.
Giải thích:
Tại các khe 3, 4 và 5:
Q1 (Latch) là thiết bị lưu trữ tích cực theo mức, cho phép ngõ ra thay đổi bám theo ngõ vào hoặc giữ mức tín hiệu tùy thuộc vào trạng thái mức của CLK.
Q2 (Flip-Flop) là thiết bị lưu trữ tích cực theo cạnh (sườn lên) nên hoàn toàn không phản ứng với các mức duy trì hay sườn xuống của CLK trong khoảng thời gian này. Sự khác biệt về cách đáp ứng tín hiệu điều khiển tạo ra độ lệch trạng thái kéo dài qua các khe 3, 4 và sang tận khe 5.
Tại các khe 11, 12 và 13:
Tương tự ở chu kỳ xung nhịp tiếp theo, khi xung nhịp CLK tác động kết hợp với dữ liệu D thay đổi, Latch tiếp tục duy trì tính trong suốt theo mức làm ngõ ra Q1 thay đổi theo.
Ngược lại, Flip-Flop do cơ chế chốt theo cạnh xung nhịp nên bỏ qua các biến động mức trung gian và giữ nguyên giá trị cũ, dẫn đến sự khác biệt Q1 khác Q2 kéo dài từ khe 11, 12 sang đến tận khe 13.
Nguyên nhân trong khoảng CLK = 1 mà ngõ vào D đổi nhưng chỉ có Q1 đổi còn Q2 thì không:
Q1 (Latch) hoạt động theo mức (Level-triggered), nên khi CLK duy trì ở mức 1 thì cổng cho phép mở hoàn toàn, khiến mọi thay đổi của D đều được cập nhật ngay lập tức vào Q1.
Q2 (Flip-Flop) hoạt động theo cạnh (Edge-triggered), chỉ cho phép lấy mẫu và thay đổi trạng thái tại đúng khoảnh khắc chuyển mạch duy nhất là sườn lên (0 lên 1). Toàn bộ thời gian sau đó khi CLK đã ổn định ở mức 1, mọi biến động của D đều bị vô hiệu hóa đối với Flip-Flop, làm cho Q2 giữ nguyên trạng thái.

C. Vì sao thanh ghi dùng flip-flop mà không dùng latch?

DỰNG MẠCH **Bước 3.4.** Trong circuit TN3, dựng hai chuỗi hai tầng: **(i)** hai DLATCH nối tiếp (Q của latch 1 vào D của latch 2), cả hai có E = CLK; **(ii)** hai D Flip-Flop nối tiếp cùng CLK. Cả hai chuỗi dùng chung ngõ vào D.

DỰ ĐOÁN Ban đầu mọi Q = 0. Mỗi **xung** là CLK: 0 → 1 → 0, D giữ nguyên trong xung. Điền Q1, Q2 (viết dạng "Q1 Q2") sau mỗi xung, rồi đo.

Bảng 12. Chuỗi hai tầng: latch và flip-flop

Xung	D	2 latch: dự đoán	2 latch: đo	2 flip-flop: dự đoán	2 flip-flop: đo
1	1	1 1	1 1	1 0	1 0
2	0	0 0	0 0	0 1	0 1
3	0	0 0	0 0	0 0	0 0

Câu 3.5 (Đánh giá). Từ Bảng 12, dữ liệu đi qua mấy tầng trong **một** xung ở mỗi cấu hình? Từ đó dự đoán điều gì xảy ra với lệnh **R1 ← R1 + R0** (ngõ ra ALU quay lại ngõ vào chính thanh ghi R1)

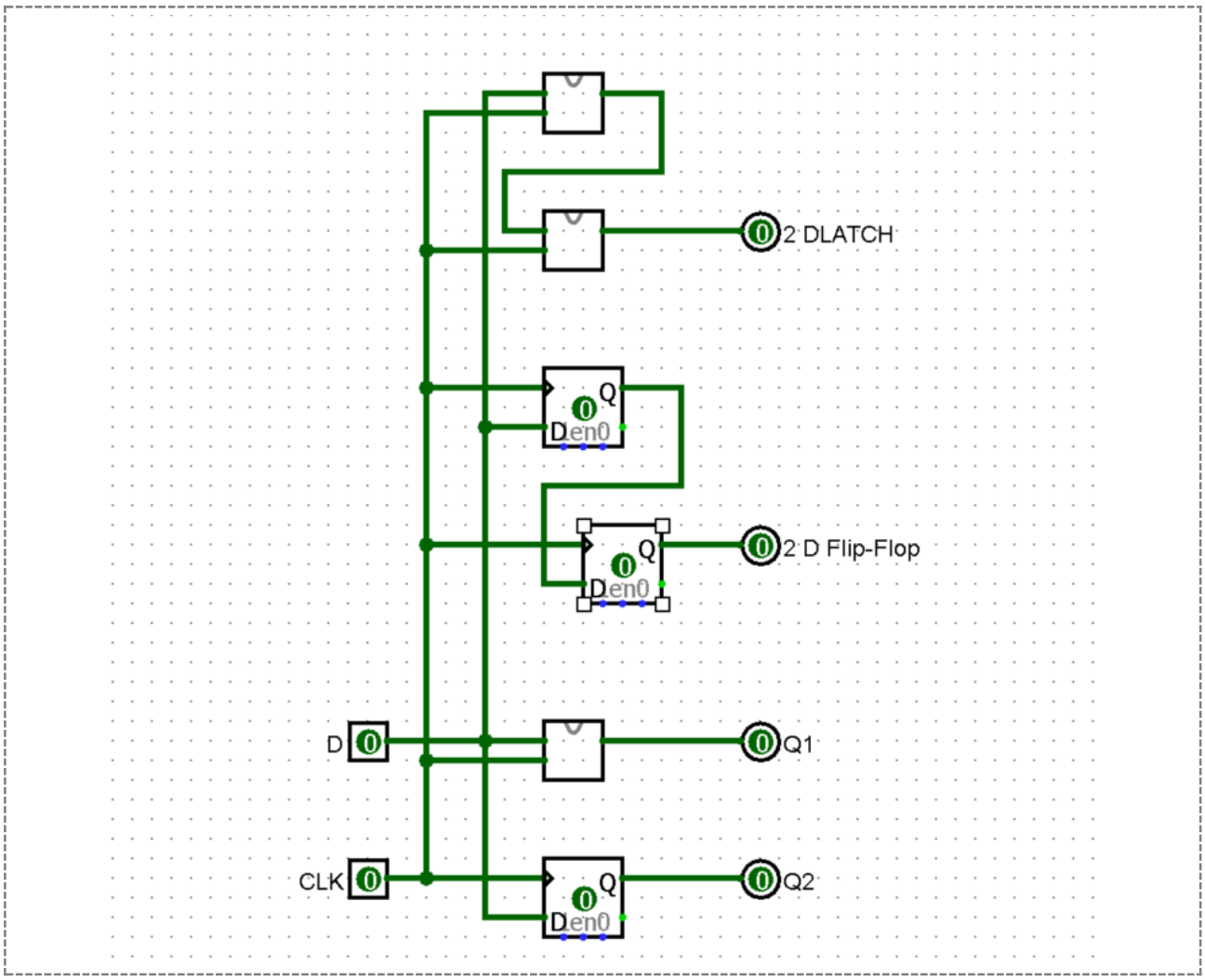
khi R1 được làm bằng latch trong suốt lúc CLK = 1, và khi R1 làm bằng flip-flop. (Sinh viên sẽ kiểm chứng ở Thí nghiệm 4.)

Số tầng dữ liệu đi qua trong một xung:
Cấu hình 2 LATCH: Dữ liệu đi qua 2 tầng.

Giải thích: Do LATCH hoạt động theo mức và ở trạng thái trong suốt khi CLK = 1, cho phép dữ liệu truyền thẳng qua cả hai tầng liên tiếp trong cùng một chu kỳ xung nhịp.

Cấu hình 2 Flip-Flop: Dữ liệu đi qua 1 tầng.
Giải thích: Do Flip-Flop chỉ cập nhật dữ liệu tại thời điểm sườn lên của xung nhịp, nên dữ liệu dịch chuyển từ tầng 1 sang tầng 2 phải chờ đến chu kỳ xung nhịp tiếp theo.

Dự đoán với R1 ← R1 + R0:
Khi R1 làm bằng LATCH: Giá trị bị thay đổi liên tục, không ổn định và dẫn đến sai số nghiêm trọng.
Giải thích: Do Latch trong suốt suốt thời gian CLK = 1, kết quả từ ngõ ra ALU sẽ lập tức lọt ngược lại ngõ vào, tiếp tục cộng dồn tạo thành vòng lặp dao động liên tục (hiện tượng chạy đua) trong cùng một xung.
Khi R1 làm bằng Flip-Flop: Phép tính thực hiện chính xác, ổn định và cập nhật giá trị đúng thời điểm.
Giải thích:
Vì Flip-Flop cách ly ngõ vào và ngõ ra, chỉ chốt kết quả phép tính của ALU đúng một lần tại sườn lên của xung nhịp, ngăn chặn triệt để hiện tượng phản hồi lặp vô hạn.

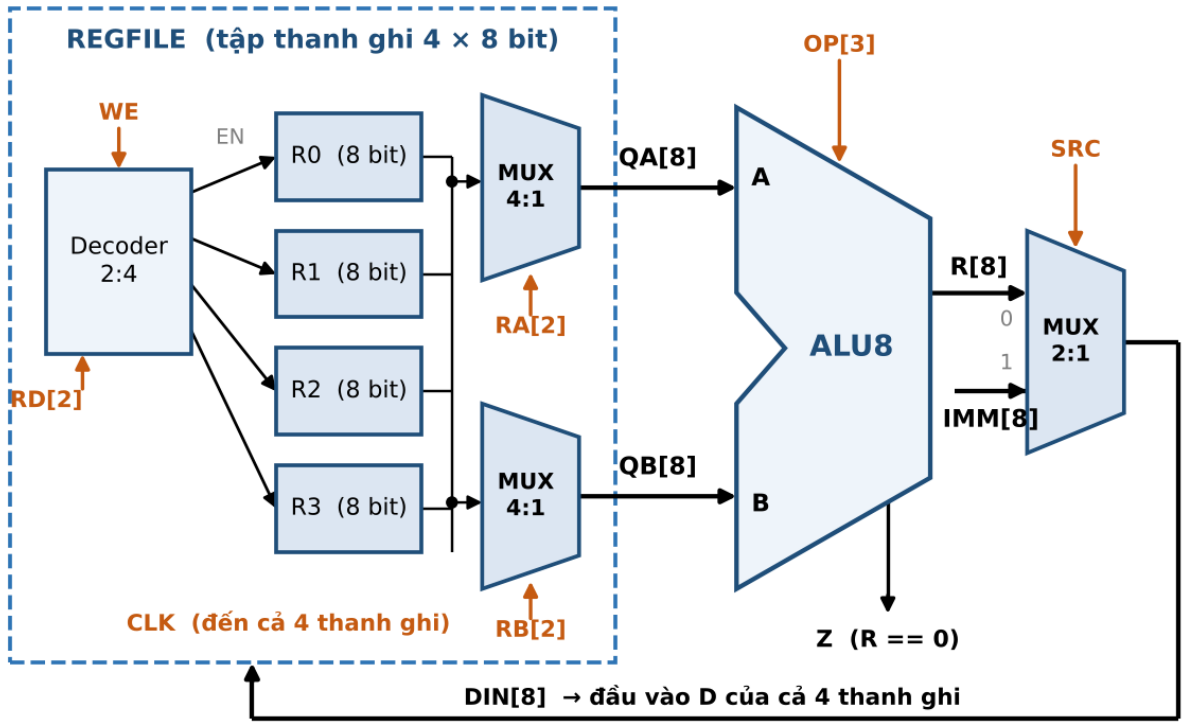

2 DLATCH
Q
Den0
Q
Den0
2 D Flip-Flop
D
Q1
CLK
Q
Den0
Q2

THÍ NGHIỆM 4 — Tập thanh ghi và đường dữ liệu 8 bit

Thời lượng gợi ý: 60 phút · Circuit: **REGFILE, DATAPATH** · Điểm: 3,0

Tình huống

Slide Chương 5 minh hoạ lệnh **C = A + B**: đưa A, B từ bộ nhớ vào **tập thanh ghi**, **ALU** tính, kết quả ghi lại vào tập thanh ghi. Sinh viên ghép các khối đã dựng thành **đường dữ liệu** của một CPU thu nhỏ và điều khiển nó bằng tay: mỗi "lệnh" là một bộ tín hiệu điều khiển cộng với một xung CLK.



Hình 3. *Đường dữ liệu mini: REGFILE (decoder 2:4 + 4 thanh ghi + 2 MUX đọc) → ALU8 → MUX chọn nguồn ghi (SRC).*

A. Tập thanh ghi 4 × 8 bit

Slide Chương 5 vẽ tập thanh ghi với một cổng đọc. ALU cần **hai toán hạng cùng lúc** nên REGFILE ở đây có **hai cổng đọc** (RA, RB) và **một cổng ghi** (RD).

DỰNG MẠCH **Bước 4.1.** Trong circuit **REGFILE**: Input Pin **CLK, WE** (1 bit), **RD** (2 bit, địa chỉ ghi), **DIN** (8 bit), **RA, RB** (2 bit, địa chỉ đọc); Output Pin **QA, QB** (8 bit) và **R0 … R3** (8 bit, để theo dõi). Dùng: một **Decoder 2:4** (Select = RD; nối WE vào ngõ Enable nếu có, hoặc dùng 4 cổng **Multiplexer 4:1** 8 bit chọn Rk theo RA và theo RB.

DỰ ĐOÁN Với các bước 5–8 của Bảng 13, điền cột "R1 dự đoán" **trước** khi Poke. Mỗi bước kết thúc bằng một xung CLK (0 → 1 → 0) trừ khi ghi chú khác.

QUAN SÁT Ghi R0 … R3 (hex) sau từng bước.

Bảng 13. Ghi và đọc trên REGFILE

Bước	Thao tác	R1 dự đoán	R0	R1	R2	R3
0	Vừa Reset Simulation (WE = 0)	0x00	0x00	0x00	0x00	0x00

Bước	Thao tác	R1 dự đoán	R0	R1	R2	R3
1	RD = 0, WE = 1, DIN = 0x11, xung CLK	–	0x11	0x00	0x00	0x00
2	RD = 1, WE = 1, DIN = 0x22, xung CLK	–	0x11	0x22	0x00	0x00
3	RD = 2, WE = 1, DIN = 0x33, xung CLK	–	0x11	0x22	0x33	0x00
4	RD = 3, WE = 1, DIN = 0x44, xung CLK	–	0x11	0x22	0x33	0x44
5	RD = 1, **WE = 0**, DIN = 0x99, xung CLK	0x22	0x11	0x22	0x33	0x44
6	RD = 1, WE = 1, DIN = 0x99, **CLK vẫn = 0**	0x22	0x11	0x22	0x33	0x44
7	CLK: 0 → 1 (giữ CLK = 1)	0x99	0x11	0x99	0x33	0x44
8	CLK: 1 → 0	0x99	0x11	0x99	0x33	0x44

Câu 4.1 (Hiểu). Vì sao cần tín hiệu WE ngoài CLK? Điều gì xảy ra nếu bỏ WE (mỗi xung CLK đều ghi DIN vào thanh ghi do RD chọn) khi ta chỉ muốn **so sánh** hai thanh ghi mà không muốn ghi kết quả?

Cần tín hiệu WE (Write Enable) kết hợp với xung CLK để kiểm soát chính xác thời điểm nào được phép ghi dữ liệu vào tập thanh ghi, tránh việc thanh ghi bị ghi đè dữ liệu ngoài ý muốn mỗi khi có xung nhịp CLK.
Nếu bỏ tín hiệu WE (nghĩa là mọi xung CLK đều thực hiện ghi), thì trong các tác vụ chỉ dùng để đọc, trích xuất dữ liệu hoặc so sánh hai thanh ghi (ví dụ: ALU nhận dữ liệu từ QA và QB để so sánh), mỗi khi có chu kỳ xung nhịp đi qua, dữ liệu ngõ vào DIN sẽ vô tình bị ghi đè lên thanh ghi đang được chọn bởi RD. Điều này làm mất dữ liệu cũ, sai lệch trạng thái lưu trữ của hệ thống và gây lỗi chương trình.

Câu 4.2 (Phân tích). Khi RD = RA = 2, WE = 1 và DIN thay đổi: QA đổi **ngay khi DIN đổi** hay **chỉ sau cạnh lên**? Vì sao điều này quan trọng cho lệnh R2 ← R2 + R1 (liên hệ Bảng 12)?

Thời điểm thay đổi của QA:
QA không đổi ngay khi DIN đổi mà chỉ thay đổi sau cạnh lên (sườn lên) của xung nhịp CLK (khi WE = 1).
Cách kiểm tra: Quan sát ngõ ra QA trong Logisim, giá trị chỉ cập nhật khi xung nhịp chuyển từ mức 0 lên 1.
Ý nghĩa quan trọng đối với lệnh R2 ← R2 + R1:
Giúp duy trì ổn định giá trị ban đầu của R2 tại ngõ vào QA trong suốt quá trình ALU thực hiện phép tính, tránh việc bị ghi đè dữ liệu mới (DIN) làm sai lệch toán hạng đầu vào.
Cách kiểm tra: ALU tính toán xong với dữ liệu cũ của R2, sau đó giá trị kết quả mới chính thức được ghi vào thanh ghi ở chu kỳ xung nhịp tiếp theo.

B. Ghép đường dữ liệu

DỰNG MẠCH **Bước 4.2.** Trong circuit **DATAPATH**: dùng mạch con REGFILE, mạch con ALU8 và một Multiplexer 2:1 (8 bit) chọn DIN: **SRC = 0 → R** (kết quả ALU), **SRC = 1 → IMM** (hằng

số). Input Pin: **IMM** (8 bit), **SRC**, **OP** (3 bit), **RA, RB, RD** (2 bit), **WE**, **CLK**; Output Pin: **R0 … R3**, **R**, **Z**. Nối theo Hình 3 (A ← QA, B ← QB).

> **Quy ước "một lệnh"**
> - Một lệnh = đặt đúng các tín hiệu điều khiển (SRC, OP, RA, RB, RD, WE, IMM) **rồi** tạo một xung CLK (0 → 1 → 0).
> - Lệnh **nạp hằng**: SRC = 1. Lệnh **ALU**: SRC = 0. Lệnh **chỉ kiểm tra** (không ghi): WE = 0, đọc cờ Z.
> - Ô không dùng đến ghi dấu "–".

DỰ ĐOÁN Điền **từ vựng điều khiển** của 5 lệnh ở Bảng 14 (số thanh ghi viết dạng thập phân, OP dạng nhị phân, IMM dạng hex).

Bảng 14. Tín hiệu điều khiển của từng lệnh

Lệnh	SRC	OP	RA	RB	RD	WE	IMM
(1) R0 ← 0x05	1	-	-	-	0	1	0x05
(2) R1 ← 0x07	1	-	-	-	1	1	0x07
(3) R2 ← R0 + R1 (C = A + B ở slide)	0	000	0	1	2	1	-
(4) R3 ← R2 − R0	0	010	2	0	3	1	-
(5) So sánh R3 với R1 (không ghi)	0	010	3	1	-	0	-

QUAN SÁT Chạy lần lượt 5 lệnh (từ trạng thái Reset Simulation) và ghi R0 … R3 (hex) và cờ Z sau mỗi lệnh.

Bảng 15. Kết quả chạy chương trình (1)–(5)

Sau lệnh	R0	R1	R2	R3	Z
(1)	0x05	0x00	0x00	0x00	–
(2)	0x05	0x07	0x00	0x00	–
(3)	0x05	0x07	0x0C	0x00	0
(4)	0x05	0x07	0x0C	0x07	0
(5)	0x05	0x07	0x0C	0x07	1

Câu 4.3 (Hiểu). Lệnh (5) không ghi gì nhưng vẫn hữu ích. Z = 1 cho biết điều gì? So với quy tắc "MSB" ở Thí nghiệm 2, kết luận "bằng nhau" từ Z có luôn đúng không? Vì sao?

Ý nghĩa của Z = 1: Cờ Z = 1 cho biết kết quả phép trừ của ALU bằng 0, nghĩa là hai toán hạng được đem ra so sánh có giá trị bằng nhau.
Tính đúng đắn của kết luận "bằng nhau" từ cờ Z so với quy tắc MSB: Không phải lúc nào cũng đúng đối với trường hợp số có dấu. Cờ Z chỉ xác định toàn bộ các bit kết quả bằng 0 mà không xét đến dấu và hiện tượng tràn số (overflow), do đó việc kết luận hai số có dấu bằng nhau chỉ dựa vào cờ Z là chưa đủ chính xác trong mọi trường hợp.

TỔNG KẾT VÀ NỘP BÀI

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File **IT012_Lab02_<MSSV>.circ** có đủ các circuit **FA, ADDSUB4, ALU8, LT, DLATCH, DFF_MS, TN3, REGFILE, DATAPATH**; mọi Input/Output Pin đều có nhãn.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với mạch nộp; sau khi sửa mạch con (FA, ALU8…) đã kiểm tra lại các mạch dùng nó.
☑	Đặt tên file báo cáo: **IT012_Lab02_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên.

Phản tư

Câu R1. Từ Thí nghiệm 1 và 2, Sinh viên rút ra điều gì về sự khác nhau giữa "kết quả **đúng về mặt bit**" và "kết quả **đúng về mặt ý nghĩa**" (có dấu / không dấu)? Phần cứng giúp phần mềm phân biệt hai điều đó bằng cách nào?

Qua Thí nghiệm 1 và 2, em thấy một kết quả có thể đúng về mặt bit nhưng chưa chắc đúng về mặt ý nghĩa. Cùng một dãy bit có thể là hai giá trị khác nhau tùy xét số không dấu hay có dấu (bù 2), ví dụ 0xFF là 255 hoặc −1.
Phần cứng chỉ cộng/trừ trên các bit, không biết đó là số có dấu hay không dấu. Nó giúp phần mềm phân biệt bằng các cờ trạng thái: cờ C (carry/borrow) báo tràn khi xét số không dấu, cờ V (overflow) báo tràn khi xét số có dấu, và phần mềm chọn cờ phù hợp với kiểu dữ liệu đang dùng.

Câu R2. Từ Thí nghiệm 3 và 4: trong DATAPATH, khối nào là mạch **tổ hợp**, khối nào là mạch **tuần tự**? Chỉ ra một vòng hồi tiếp (từ ngõ ra thanh ghi, qua ALU, quay về ngõ vào thanh ghi) và giải thích vì sao vòng này không gây hỗn loạn.

Trong DATAPATH, ALU, MUX và bộ giải mã là mạch tổ hợp vì ngõ ra chỉ phụ thuộc ngõ vào hiện tại. Các thanh ghi (REGFILE) là mạch tuần tự vì lưu trạng thái và chỉ cập nhật theo xung CLK.
Một vòng hồi tiếp: ngõ ra thanh ghi R1 → ALU → MUX 2:1 → DIN → ngõ vào D của thanh ghi. Vòng này không gây hỗn loạn vì thanh ghi chỉ lấy mẫu DIN tại cạnh lên của CLK (và khi WE = 1). Giữa hai xung, giá trị lưu trong thanh ghi không đổi nên ALU nhận ngõ vào ổn định, kết quả tính xong rồi mới được chốt vào một lần.

Câu R3. Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Lỗi gặp phải là ngõ ra Decoder trong REGFILE bị đặt Disabled Output = Floating, nên khi WE = 0 các thanh ghi vẫn bị ghi theo xung CLK. Em phát hiện khi chạy Bảng 13 bước 5 (WE = 0) mà R1 vẫn bị đổi thành 0x99 thay vì giữ 0x22, đồng thời thấy dây ngõ ra Decoder có màu xanh dương. Em sửa thành Disabled Output = Zero. Bài học là phải chạy thử đối chiếu với giá trị dự đoán sau mỗi bước, và kiểm tra cả trường hợp "không ghi", không chỉ trường hợp ghi

Câu R4 (mở, không chấm). Để DATAPATH chạy được một chương trình có vòng lặp hay rẽ nhánh ("nếu Z = 1 thì làm A, ngược lại làm B") mà **không cần người đặt tín hiệu bằng tay**, Sinh viên cần thêm những khối nào? Ghi ý tưởng, chưa cần vẽ mạch.

Cần thêm các khối điều khiển: bộ đếm chương trình (PC) giữ địa chỉ lệnh hiện tại, bộ cộng +1 để tăng PC; bộ nhớ lệnh chứa chương trình; bộ giải mã lệnh/mạch điều khiển để tạo các tín hiệu SRC, OP, RA, RB, RD, WE, IMM. Với rẽ nhánh, thêm MUX chọn PC kế tiếp giữa PC + 1 và địa chỉ nhánh, điều khiển bởi cờ Z và lệnh rẽ nhánh (nếu Z = 1 thì nhảy, ngược lại chạy tiếp). Nhờ vậy CPU tự chạy tuần tự, rẽ nhánh và lặp mà không cần đặt tín hiệu bằng tay.

Tài liệu tham khảo

- Slide bài giảng Chương 4 – Mạch số và Chương 5 – Ứng dụng mạch số (IT012, Khoa Kỹ thuật Máy tính, UIT).
- Slide bài giảng Chương 2 – Biểu diễn thông tin trong máy tính (số có dấu bù 2) và Chương 3 – Đại số Boole (bìa Karnaugh, chi phí C(B)).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.